

VIDEO 1 - STACK DATA STRUCTURE

Use this script with the Stack presentation. Speak naturally rather than reading too quickly; pause while pointing to diagrams and while demonstrating the C++ program.

Opening / Title Slide

Hello everyone, my name is Rushikesh Nayak. In this first part of my BLA Lab 01, I am going to explain the Stack data structure. I will cover the LIFO principle, the main Stack operations, important conditions such as overflow and underflow, and how a Stack can be implemented using a static array in C++. After the concept explanation, I will also demonstrate the program so we can see how these operations work in practice.

Before I begin, I would like to thank Professor Victor Govindaswamy for his guidance throughout this course. His explanations, examples, quizzes, and programming activities are helping me understand Data Structures step by step and connect the theory with practical C++ programming.

Stack Data Structure and LIFO Principle

A Stack is a linear data structure where insertion and removal happen from only one end, which is called the TOP. The most important rule of a Stack is LIFO, which stands for Last In, First Out. This means the last value added to the Stack is the first value that will be removed.

A simple way to understand this is to think about a stack of trays. If I place one tray on top of another, the last tray I place is normally the first tray I take away. A computer Stack follows the same basic idea.

LIFO Example

In this example, I add 10 first, then 20, and finally 30. Because 30 was added last, it becomes the TOP element. If I perform a Pop operation, 30 is removed first. After that, 20 becomes the new TOP.

So the order is easy to remember: 10 entered first, 20 entered second, 30 entered last, and 30 leaves first. That is exactly why this structure is called Last In, First Out.

Main Stack Operations - Push, Pop and Peek

Now I will explain the three main operations. Push is used to add a new value to the TOP of the Stack. For example, if 10 and 20 are already stored and I push 30, then 30 is placed above 20 and becomes the new TOP.

Pop does the opposite. It removes the current TOP element. If 30 is at the TOP and I call Pop, 30 is removed and 20 becomes the new TOP.

Peek, sometimes called Top, only lets us view the current TOP value. It does not remove anything. So if the TOP contains 30, Peek returns 30 and the Stack stays unchanged.

Push / Pop Diagram

This diagram shows the same process visually. Before the Push operation, 20 is at the TOP. When I push 30, 30 is placed above it and becomes the new TOP. When I Pop 30, that value is removed and 20 becomes the TOP again.

The important difference is that Push adds, Pop removes, and Peek only checks the TOP without changing the Stack.

Checking and Displaying a Stack

There are also useful checks that make a Stack program safer. isEmpty checks whether the Stack currently contains zero elements. We should check this before Pop or Peek because there is no valid TOP element if the Stack is empty.

isFull is important when we use a fixed-size array. It checks whether every available position is already occupied. Display shows the values currently stored in the Stack, normally starting from the TOP and moving downward.

Why These Checks Matter

These checks help prevent invalid operations. Before adding another element, the program can check whether the Stack is full. Before removing or viewing an element, it can check whether the Stack is empty.

This is especially important with a static array because its capacity is fixed. The program should not try to access a position that does not exist.

Stack Overflow and Underflow

Two important error conditions are Stack Overflow and Stack Underflow. Overflow happens when we try to Push another element into a Stack that is already full. For example, if the capacity is three and the Stack already contains 10, 20, and 30, there is no available position for 40.

Underflow is the opposite situation. It happens when we try to Pop from an empty Stack. There is no value available to remove.

My easy way to remember this is: Overflow means full plus trying to Push, while Underflow means empty plus trying to Pop.

Static Array Implementation

A Stack can be implemented using a static array. A static array has a fixed size, so the maximum capacity is decided in advance. We also use a variable called top to keep track of the index of the current TOP element.

In this example, the array capacity is five. Values 10, 20, and 30 are stored at indexes zero, one, and two, so top is equal to two. If I Push 40, it is stored at index three and top becomes three. If I Pop again, 40 is removed logically and top moves back to two.

Advantages and Limitations

An array-based Stack is simple to understand and straightforward to implement. Push and Pop work only with the TOP, so these operations are easy to manage.

The limitation is that a static array cannot automatically grow. If all positions are occupied, another Push causes Overflow. Also, if the maximum capacity is larger than what we actually use, some allocated positions can remain unused.

Live C++ Demonstration

Now I will move to the C++ program. My program uses a fixed-size integer array and a top variable. At the beginning, top is negative one, which represents an empty Stack.

The menu allows the user to choose Push, Pop, Peek, Display, or Exit. When I select Push, the program asks for a value, moves top to the next position, and stores the value there. Pop removes the current TOP by moving top backward. Peek displays the current TOP without changing it, and Display prints the Stack from TOP to bottom.

While running the program, I will enter a few values and then use Display and Peek so we can verify the order. I will also demonstrate the full and empty conditions so we can see how the program protects against Overflow and Underflow.

Closing

This completes my explanation of the Stack data structure. Through this part of the assignment, I learned how LIFO controls the order of operations, how the TOP variable represents the current Stack position, and how a simple static array can be used to implement Push, Pop, Peek, and Display.

Thank you for watching, and again, thank you to Professor Victor Govindaswamy for the guidance and practical learning opportunities in this course.